

Fast4DGS: Accelerating Compact 4D Gaussian Splatting Reconstruction from Monocular Videos

Chang Liu, Chenyi Jin, Shangzhe Ding, Qifeng Chen, Hanggao Xin, Sheng Yang, Hongbo Fu, Shi-Sheng Huang

Abstract—High-fidelity monocular 4D Gaussian splatting (4DGS) remains computationally expensive, because existing methods optimize large numbers of spatially and temporally redundant primitives for each scene. We present Fast4DGS, an efficient framework that reconstructs dynamic scenes using a spatiotemporally compact Gaussian representation. This representation separates static and dynamic Gaussians and models the motion of each dynamic primitive with a low-dimensional parametric trajectory, thereby reducing the number of learnable time-varying parameters. To enable rapid convergence, a motion-aware initialization fits these trajectories within overlapping temporal windows, yielding geometrically compact and temporally coherent primitives over long sequences. A locally compact pruning strategy further merges redundant primitives while preserving their spatial coverage through anisotropic covariance adaptation. Together, these components reduce representational redundancy before photometric optimization, lowering optimization cost and memory consumption while preserving high-fidelity dynamic reconstruction. Experiments on DyCheck iPhone, DAVIS, and NeRF-DS demonstrate consistent gains in optimization efficiency with competitive or superior reconstruction quality. On challenging DyCheck iPhone, Fast4DGS achieves 24.83 dB PSNR at 0.432 seconds per-frame of training speed, surpassing Instant4D by 0.55 dB while delivering a $3.27\times$ speedup and reducing peak GPU memory from 16.39 GB to 2.18 GB.

Index Terms—Dynamic Scene Reconstruction, 4D Gaussian Splatting, Compact Scene Representation, Motion Representation

I. INTRODUCTION

RECONSTRUCTING dynamic scenes in 4D from videos has long been an important research problem in computer vision and graphics [2]–[4]. This capability underpins dynamic content creation for a broad range of computer graphics, computer vision and VR/AR applications [5], including immersive teleconferencing [6]–[8], virtual avatars [9]–[11], volumetric video [12], [13], and dynamic scene simulation [14]–[17]. While multiview 4D reconstruction benefits from cross-view consistency cues [18]–[21], monocular 4D reconstruction lacks such constraints, making it particularly challenging to recover scene geometry and motion from inherently ambiguous observations [22].

Neural scene representations have substantially advanced dynamic view synthesis from monocular videos. Dynamic

NeRF methods achieve impressive reconstruction quality, but their ray-marching-based rendering and per-scene optimization incur considerable computational cost [23]–[25]. The emergence of 3D Gaussian Splatting (3DGS) [26] and its dynamic extensions [27]–[29] has greatly improved rendering efficiency by replacing dense volumetric sampling with explicit Gaussian primitives. Nevertheless, fast rendering does not necessarily imply fast reconstruction: recent monocular 4DGS methods still require expensive per-scene optimization over a large number of Gaussian primitives and time-varying parameters, even when depth, optical-flow, or point-tracking priors are available [30]–[33]. Although feed-forward 4D reconstruction methods amortize reconstruction over large-scale pretraining and enable second-level inference [34]–[37], this efficiency often comes with a scene-specific fidelity gap. Without sufficient optimization, existing feed-forward methods may struggle to preserve fine-grained appearance, complex motion, and object identity in challenging in-the-wild videos. Per-scene optimization therefore remains important when faithful reconstruction of a particular monocular video is required.

Recent per-scene acceleration methods reduce the optimization cost through spatial consolidation, temporal pruning, motion grouping, or progressive reconstruction [38]–[40]. However, aggressively removing Gaussian primitives can reduce the representation capacity and spatial coverage of dynamic regions. Existing methods therefore introduce error-guided densification, hierarchical reconfiguration, or track-guided relocation to allocate additional primitives where needed [41]–[43]. Although effective, these operations introduce additional primitives and optimization overhead, partially offsetting the computational benefit of pruning. Moreover, determining where new primitives should be allocated is particularly difficult under monocular input, where geometry, motion, and visibility estimates are inherently uncertain. Optimization-dependent pruning [39], also reduces redundancy only after part of the dense-model training cost has already been incurred, whereas directly starting from fewer dynamic Gaussians may result in incomplete spatial coverage and temporally inconsistent trajectories. Therefore, an important question remains: can spatial and temporal redundancy be reduced before photometric optimization while retaining the scene coverage and motion expressiveness required for high-quality monocular reconstruction?

In this paper, we propose a new mechanism, i.e., Fast4DGS, to substantially accelerate the per-scene optimization process of monocular 4DGS reconstruction, while preserving high-fidelity dynamic reconstruction quality simultaneously. Unlike optimizing the redundant dynamic offsets as previous

Chang Liu, Chenyi Jin, Shangzhe Ding, and Shi-Sheng Huang are with the School of Artificial Intelligence, Beijing Normal University, Beijing, China (e-mail: huangss@bnu.edu.cn). Corresponding author: Shi-Sheng Huang.

Qifeng Chen, Hanggao Xin and Sheng Yang are with Assisted and Intelligent Driving, NIO Inc., China. (e-mail: cqf7419@gmail.com)

Hongbo Fu is with the Division of Arts and Machine Creativity, The Hong Kong University of Science and Technology, Hong Kong, China. E-mail: hongbofu@ust.hk.



Fig. 1. **Fast4DGS: Fast, compact, and high-fidelity monocular 4D reconstruction.** Left: on the challenging *paper-windmill* sequence, our method recovers sharp, coherent blades with fewer motion artifacts than prior methods, while requiring less optimization time and GPU memory. Right: the quality-efficiency comparison on DyCheck iPhone [1], where marker area represents peak GPU memory. Fast4DGS occupies the favorable upper-left region, achieving the highest reconstruction quality and the shortest per-frame training time with a compact memory footprint. Supplementary video for additional dynamic results.

works [40], our primary idea is to formulate dynamic Gaussian primitives maximally to be *compact* in both spatial and temporal domain, which brings minimized number of optimization parameters for fast per-scene optimization while maintaining reconstruction quality. Specifically, we propose to represent dynamic scenes as spatiotemporally *compact* 4DGS primitives. To make the representation compact, we first decompose compact 4DGS primitives into static and dynamic Gaussians using an accurate static-dynamic identification, and then explicitly model the dynamic motion as a low-dimensional parametric trajectory [28], [29] with minimized motion parameters to be optimized. One drawback for the explicit based 4DGS [28], [29] is the quality degradation during long-range sequence. Thus we propose a motion-aware initialization to create geometrically *compact* and temporally *coherent* candidates across long-range sequences, which in turn improve their spatial compact for fast convergence. Moreover, we also provide a locally compact pruning strategy to further reduce the temporally redundancy of 4DGS primitives, while effectively accelerating the 4DGS reconstruction processing. By integrating those strategies within a whole optimization, we achieve *compact* and temporally non-redundant 4DGS primitives for fast and high-fidelity reconstruction from monocular videos.

To evaluate the effectiveness of our Fast4DGS, we perform extensive evaluation on public dataset including DyCheck iPhone [1], DAVIS [44], and NeRF-DS [45], and make comparison with previous state-of-the-art 4DGS baselines like: (1) monocular 4DGS acceleration approaches including Instant4D [38], SpeDe3DGS [39] and ReCon-GS (monocular) [42], (2) other 4DGS baselines including MoSca [31], 4DGS [27], and STG [28]. By quantitative and qualitative evaluation, our Fast4DGS can achieve much faster training speed than these 4DGS baselines. Specifically, Fast4DGS achieves $3.27\times$, $2.58\times$, $1.68\times$ faster than Instant4D on DyCheck iPhone, DAVIS, and NeRF-DS respectively, where Instant4D being the previous state-of-the-art baseline. Besides, Fast4DGS also achieves consistently better dynamic rendering quality than these 4DGS baselines across three datasets. This shows that Fast4DGS tracks the best performance for the balance between training speed and reconstruction quality, especially with much faster training speed benefit from the compact 4DGS acceleration mechanism proposed.

II. RELATED WORK

A. Dynamic 3D Reconstruction

The early work of dynamic 3D reconstruction focus on non-rigid dynamic fusion [46] in real-time to a wide range of objects and scenes. With the aid of parameter model such as SMPL [47], human-centric dynamic reconstruction [48], [49] has achieved impressive reconstruction quality. Another line of research adopts depth-based rendering techniques [50] to continuously improve the rendering quality [6], spatial-temporal consistency [20], [25] and efficiency [51], [52] of dynamic reconstruction.

NeRF [53] and 3D Gaussian splatting [26] have become two dominant paradigms for dynamic 3D scene reconstruction, enabling high-fidelity novel-view synthesis from visual observations. To handle time-varying motions, dynamic NeRF variants [23], [24], [54], [55] achieve promising reconstruction quality for multi-view dynamic inputs. Nevertheless, NeRF-based dynamic methods suffer from heavy computational overhead caused by ray-marching sampling. Extending 3DGS to dynamic scenarios yields 4D Gaussian Splatting (4DGS) [27], [56]–[58], which embeds temporal deformation into Gaussian primitives to model motion across frames. There are mainly two types of 4DGS approaches, which differ in whether they model the deformation field implicitly [21], [59] or explicitly [28], [29]. However, under monocular settings, insufficient multi-view constraints easily lead to distorted shapes, artifacts and unstable reconstruction results. Our work is inspired by these previous dynamic 3D reconstruction approaches, but aims at high-fidelity dynamic reconstruction from monocular videos in fast training speed.

B. Monocular 4D Gaussian Splatting

Compared with multiview 4D reconstruction that benefits from sufficient geometric constraints, monocular 4DGS faces inherent challenges due to insufficient spatial cues, temporal ambiguity, and ill-posed depth estimation, which severely restrict its practical application in casual video capture scenarios. Existing monocular 4DGS methods mainly optimize dynamic Gaussian primitives by embedding temporal deformation fields to model frame-wise scene motion. The Shape-of-Motion (SoM) [30] probably for the first time parametrizes dynamic 3D Gaussian motion via low-dimensional SE(3) trajectory

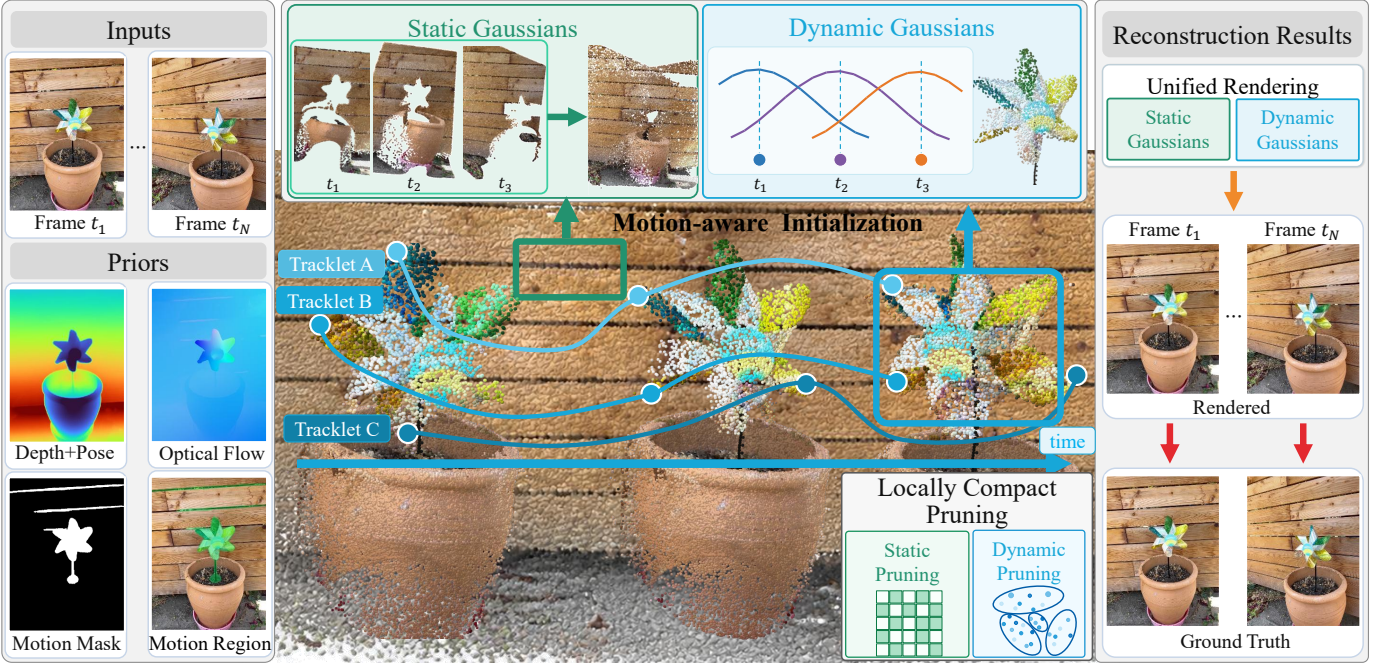


Fig. 2. Overview of Fast4DGS. Given monocular video sequence as input and thereafter reprocessing for depth, camera pose, optical flow and motion mask priors, Fast4DGS performs the reconstruction by optimizing the dynamic scene as compact 4DGS primitive set. Compact 4DGS set adaptively contains static and dynamic Gaussians, where the former keeps fixed and the latter explicitly models the dynamic as motion parameters, with minimized learning parameters for fast training. A motion-aware initialization builds up geometrically compact and temporally coherent 4DGS candidates, and locally compact pruning further reduce the redundancy for the compact 4DGS optimization.

basis functions for temporally consistent 4D scene reconstruction. MoSca [31] presents a novel 4D motion scaffold representation that anchors and regularizes dynamic 3D Gaussians for robust, high-quality dynamic scene reconstruction from casual monocular videos. MoDGS [32] leverages single-view depth priors for impressive dynamic scenes reconstruction. Recently, RU4D-SLAM [60] integrates uncertainty-aware reweighting into 4D-GS SLAM to jointly perform pose tracking and dynamic 4D mapping for degraded real-world inputs. Other works continue to improve the monocular 4D reconstruction with depth [33], flow [61], semantics [58] priors or multiple geometric cues [62]. Although achieving efficient rendering and flexible dynamic modeling results, one major drawback of these monocular 4DGS is the excessive per-scene optimization processing, bringing heavy memory overhead and limited generalization ability across diverse dynamic scenes. Different from previous monocular 4DGS approaches, we propose to optimize 4D Gaussian primitives to be compact, and effectively prune redundant 4DGS primitives for much faster training speed.

C. 4D Gaussian Splatting Acceleration

Inspired by 3DGS acceleration framework like PUP-3D [63], SpeedySplat [64], recent works introduce Gaussian primitive pruning based acceleration strategies for more efficient 4DGS reconstruction. Instant4D [38] provides a grid based 4D Gaussian pruning to effectively reduce the Gaussian primitives. Speede3DGS [39] also leverage temporal sensitive pruning with extra temporal sensitive sampling for fast 4DGS reconstruction. TrackerSplat [43] show that off-the-shelf point

tracking is effective to guide 4D Gaussians relocation, thus leading to better 4DGS reconstruction for fast motion while in efficient manner. Other works [41], [42] also introduce extra 4DGS allocation rules to make fast while accurate 4DGS reconstruction. However, most of these works focus on multiview input, which would face quality degradation for monocular scenarios. 4D-Fly [40] propagates Gaussian primitives across timesteps via an anchor-based strategy leveraging depth and tracking priors, which achieves fast dynamic scene reconstruction. However, they optimize dynamic offsets still contains temporal redundancy to limit the training speed acceleration. In contrast, we formulate the 4D Gaussian primitives in a compact form both in the spatial and temporal domains, which achieve faster training speed while maintaining high-quality monocular 4DGS reconstruction results.

III. METHOD

Given a monocular video sequence $I = \{I_t | t = 1, \dots, N\}$ of a dynamic scene, we target to reconstruct the dynamic scene as a set of compact 4DGS primitives $\mathcal{G}$ (Sec. III-A), which adaptively contains static/dynamic Gaussians with minimized learning parameters, and can be efficiently rendered using a unified static/dynamic Gaussian rasterization. A motion-aware initialization (Sec. III-B) is proposed to build up geometrically compact and temporally coherent Gaussian candidates across long-range sequence. Moreover, we propose a locally compact pruning strategy (Sec. III-C) to further reduce the 4DGS redundancy. Based on these strategies, we perform a compact 4DGS optimization (Sec. III-D) to complete the per-scene

optimization in a very fast manner. The overall pipeline is illustrated in Fig. 2.

A. Compact 4DGS

One basic idea to make the Gaussian primitive compact is to identify the static and dynamic parts within the dynamic scene, which can reduce the learning parameters size compared with alternatives that treating the whole Gaussians as dynamic [28], [29]. So we decompose compact 4DGS primitive set $\mathcal{G}$ as static Gaussian subset G^s and a dynamic Gaussian subset G^d , i.e., $\mathcal{G} = \{G^s, G^d\}$. Each static Gaussian $g_i^s \in G^s$ is parameterized by its position μ_i , anisotropic scale s_i , rotation $\mathbf{q}_i$, opacity o_i , and appearance feature $\mathbf{f}_i$, i.e., $g_i^s = \{\mu_i, s_i, \mathbf{q}_i, o_i, \mathbf{f}_i\}$, which remains temporally invariant across the input sequence. Unlike previous work [40], we explicitly model each dynamic Gaussian g_i^d as a cubic polynomial function as

$$\mu_i(t) = \mu_i^0 + \mathbf{a}_{i,1}\tau_i + \mathbf{a}_{i,2}\tau_i^2 + \mathbf{a}_{i,3}\tau_i^3, \quad \tau_i = t - c_i^m, \quad (1)$$

where μ_i^0 is the Gaussian center at the motion reference time c_i^m , and $\mathbf{a}_{i,1}$, $\mathbf{a}_{i,2}$, and $\mathbf{a}_{i,3}$ are its learnable three-dimensional motion coefficients. Compared with previous works [40], this continuous representation requires only a small number of parameters while supporting nonlinear motion. The orientation of each dynamic Gaussian is also allowed to vary over time through a learnable angular-motion parameter.

To control the temporal contribution of each dynamic Gaussian, we associate it with a learnable temporal radial basis function $w_i(t) = \exp\left(-\left(\frac{t-c_i^w}{\sigma_i}\right)^2\right)$, where c_i^w and $\sigma_i > 0$ denote its learnable temporal-weight center and scale, respectively. The motion reference c_i^m and temporal-weight center c_i^w are initialized at the same temporal anchor but remain independently learnable during optimization. Its effective opacity at time t is consequently given by $\tilde{o}_i(t) = o_i w_i(t)$. This weighting allows different dynamic primitives to take effect in different portions of the sequence, while the learnable temporal center and scale can adapt their temporal extent during optimization.

Static and Dynamic Identification. We perform a static/dynamic partition using motion and geometry priors. Firstly, for each image sequence I_t , we estimate the depth map D_t and camera pose T_t [65], and calculate adjacent-frame scene-flow S_t by lifting the estimated optical flow to 3D according to the depth map. Then we obtain a motion mask M_t by identifying the motion part if $|S_t| > \epsilon$. Thereafter, we also calculate an object level moving part mask M'_t with the aid of SAM2 [66]. Finally, we merge the motion mask M_t and moving part mask M'_t together to the final motion mask $\bar{M}_t = M_t \cup M'_t$. In this way, we identify the contents as dynamic and use to initial the dynamic Gaussian set G^d if projected onto the motion mask $\bar{M}_t$, and leave the others as static to learn the static Gaussian set G^s .

Unified Gaussian Splatting Rasterization. We also implemented a unified static and dynamic Gaussian splatting rasterization to support subsequent fast learning. Specifically, we jointly render the static and temporally transformed dynamic Gaussians primitives using differentiable Gaussian splatting:

$$\hat{I}(t) = R(G^s, G^d(t); \mathbf{K}_t, \mathbf{T}_t), \quad (2)$$

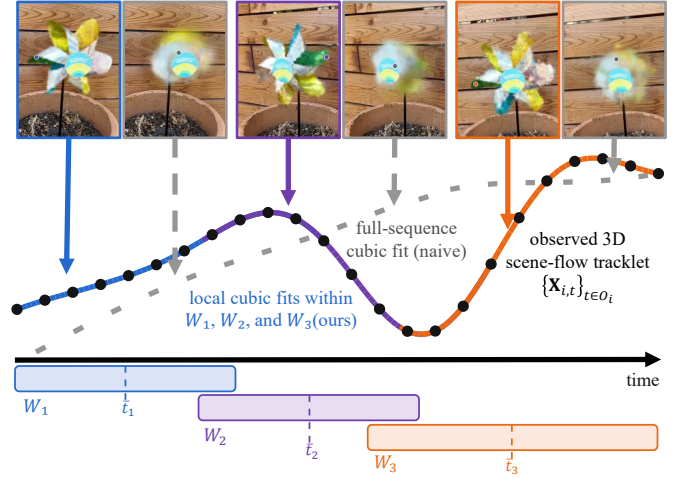


Fig. 3. Demonstration of temporally coherent initialization (curve colored) compared with naive initialization (curve gray) from a tracklet of *paper-windmill* sequence. A representative dynamic point (black dots) is observed around three temporal anchors, forming three overlapping local windows with the blue, purple, and orange curves showing the cubic trajectories fitted within the corresponding local windows. Curve gray is a single cubic fitted over the same full temporal range. For this complex motion, the global cubic visibly deviates from the observations, whereas the local cubic models remain close to the observed trajectory.

where $\mathbf{K}_t$ and $\mathbf{T}_t$ denote the camera intrinsics and pose at time t . This representation separates time-invariant geometry from temporally varying content with a unified rendering.

B. Motion-aware Initialization

To make the Gaussian primitives compact, we propose to create the compact 4DGS primitives using a motion-aware initialization. Our core idea is to pre-estimate the motion parameters of each dynamic Gaussian primitive, so that its motion trajectory is consistent with the actual tracked trajectory detected by scene flow. The pre-computed motion parameters can serve as suitable initialization, which effectively facilitate the rapid convergence of the subsequent learning process and thus enable fast reconstruction.

However, one critical challenge lies in that the quality of detected scene flow degrades over long-sequence dynamic processes. On the other hand, as like other explicit-based 4DGS representations [28], [29], the motion modeling capability of our compact 4DGS would also degrades during long-sequence motion. Therefore, simply estimating motion parameters for Gaussian primitives cannot guarantee the quality of initialization. To overcome this issue, we propose a temporally coherent initialization which preserves the compact quality during long-sequence dynamics.

Scene Flow Tracking. We perform a 3D scene flow tracking to construct 3D scene-flow tracklets, and use the tracklets to initialize the Gaussian primitives. Specifically, let $\mathbf{u}_{i,t}$ denote the image position of the i -th tracked point at timestamp t , we lift it into the 3D camera coordinate according to the depth map D_t and then transform it to the world space according to the camera pose as:

$$\mathbf{X}_{i,t}^{\text{cam}} = D_t(\mathbf{u}_{i,t})\mathbf{K}_t^{-1}\tilde{\mathbf{u}}_{i,t}, \quad \mathbf{X}_{i,t} = \mathbf{R}_t^{\text{c2w}}\mathbf{X}_{i,t}^{\text{cam}} + \mathbf{t}_t^{\text{c2w}}, \quad (3)$$

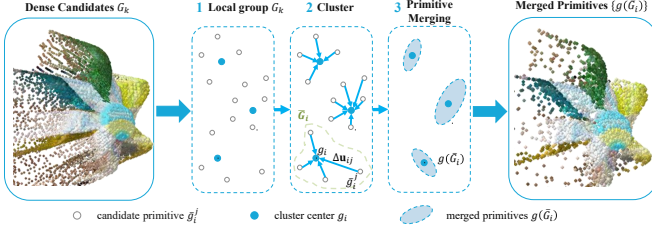


Fig. 4. Locally compact pruning of dynamic Gaussian candidates. Left: dense dynamic candidates from the *paper-windmill* example Middle: the pruning processing including primitive cluster and primitive merging from left to right. Right: Pruned results after the dynamic candidate pruning as compact set of representatives while preserving the coverage of the dynamic object.

where D_t is the depth map, $\mathbf{K}_t$ denotes the camera intrinsics, and $\tilde{\mathbf{u}}_{i,t} = (u_{i,t}, v_{i,t}, 1)^T$ is the homogeneous image coordinate. $\mathbf{R}_t^{c2w}$ and $\mathbf{t}_t^{c2w}$ are the camera-to-world rotation and translation. Tracking and lifting the corresponding pixels over time produces a sequence of 3D scene-flow tracklets $\{\mathbf{X}_{i,t}\}$ for each scene point.

Temporally Coherent Initialization. To make the compact quality of compact 4DGS primitives temporally coherent across long-sequence dynamics, we first introduce multiple temporal anchors $\{\bar{t}_k\}$ and construct a local temporal window around each anchor $W_k = \{t \mid |t - \bar{t}_k| \leq r\}$, where r is a parameter to guide the temporal window size. We also set neighboring windows overlap each other with a small size, allowing different local groups to cover shared portions of the sequence instead of partitioning the video into isolated segments.

For a tracklet assigned to the k -th temporal group, we initialize both its motion reference and temporal-weight center as $c_i^m = c_i^w = \bar{t}_k$ and use the lifted position at the anchor as its reference position μ_i^0 . Its motion coefficients are estimated by solving the fitting objective function

$$\min_{\{\mathbf{a}_{i,m}\}_{m=1}^3} \sum_{t \in W_k \cap O_i} \|\mathbf{X}_{i,t} - \mu_i(t)\|_2^2, \quad (4)$$

where O_i is the set of valid observations and $\mu_i(t)$ is defined in Eq. 1. The fitting is performed in a normalized local temporal coordinate to improve numerical conditioning, after which the coefficients are converted to the normalized timestamp.

Static Primitive Initialization. In parallel with the dynamic initialization, we initialize the static primitives from multiple temporal anchors. For each anchor frame $\bar{t}_k$, pixels with valid depth and outside the motion mask $\bar{M}_{\bar{t}_k}$ are back-projected into the world coordinate system. We aggregate the resulting static point candidates from all anchor frames and perform voxel-based fusion to remove redundant multi-view observations [38].

Discussion. Fig. 3 illustrates the visual reconstruction difference between naive 4DGS initialization and our motion-aware initialization across long-range sequence. If simply estimating the motion parameters of tracklets by fitting Eq. 4 within the whole time intervals, its motion trajectory will clearly drift a lot, leading to poor dynamic reconstruction results. In contrast, our motion-aware initialization perform the motion fitting within consequent temporal windows, which preserve

more accurate motion trajectory for much better dynamic reconstruction results. We also validate this benefit in the ablation study of Sec. IV-E.

C. Locally Compact Pruning

To further reduce the redundancy of the 4DGS primitives, we therefore propose a locally compact 4DGS pruning which is efficient while effective to preserve the dynamic quality as much as possible. Since our compact 4DGS set contains both static and dynamic primitives exhibiting different spatial and temporal characteristics, we perform the pruning separately.

Static Primitive Pruning. For the static component, we back-project pixels outside the potentially movable regions from multiple temporal anchor frames. The resulting world-space points are consolidated using voxel-based fusion. Points falling into the same voxel are replaced by one Gaussian whose position and color are obtained by aggregating their attributes. To maintain a balanced representation, the static point budget is determined according to the average ratio between static and dynamic pixels at the selected temporal anchors:

$$B_s = B_d \frac{\bar{p}_s}{\bar{p}_d}, \quad (5)$$

where B_s and B_d are the static and dynamic point budgets, while $\bar{p}_s$ and $\bar{p}_d$ denote the average static and dynamic pixel proportions, respectively. We adjust the voxel size to approximately satisfy this target budget.

Dynamic Primitive Pruning. For the dynamic component, naively applying grid-based pruning as like Instant4D [38] will destroy both the spatial geometry coverage and temporal motion dynamics, thus leading to poor 4DGS reconstruction quality. Grouping the primitives together according to motion as like SpeeDe3DGS [39] can be more effective to describe the 4DGS motion, but they rely high quality motion estimation quality which is not available for our monocular scenario. Besides, extra calculating the motion group is also time consuming which would decrease the overall training speed of the 4DGS reconstruction. Instead, we propose a clustering based dynamic pruning, which merges the 4DGS primitives within one cluster as a new 4DGS primitive with similar motion dynamics in a very efficient manner.

Primitive Clustering. Specifically, as illustrated in Fig. 4, we first perform a primitive clustering at the center timestep of each temporal window group using k-medoids [67]. Let $G_k \in G^d$ denote the dynamic candidates of the k -th local group, we cluster G_k into $|C|$ clusters, i.e., $C = \{G_i \mid i = 1, \dots, |C|\}$, and $C_k \subset D_k$ is the cluster centers. Then we prune the primitives in each cluster $\bar{G}_i \in C$ into a representative primitive $g(\bar{G}_i)$, thus reducing the dynamic primitives number from $|G_k|$ to $|C|$.

Primitive Merging. How to select the representative $g(\bar{G}_i)$ is a non-trivial problem, since it will influence the final reconstruction quality. A naive alternative is simply set the cluster center $g_i \in C_k$ as the representative, i.e., $g(\bar{G}_i) = g_i \in C_k$, but this naive alternative is not effective as we shown in the study of Sec. IV-E. Instead, we propose to merge the primitives $\bar{G}_i$ into a new one adaptively.

TABLE I

QUANTITATIVE COMPARISON ON FIVE DYCHECK IPHONE SEQUENCES UNDER OUR UNIFIED PHOTOMETRIC-SUPERVISION PROTOCOL. WE REPORT PSNR FOR EACH SCENE AND ITS AVERAGE OVER THE FIVE SCENES. THE **BEST** AND **SECOND-BEST** RESULTS ARE HIGHLIGHTED. TIME DENOTES THE MEAN TRAINING TIME PER FRAME. MEM. IS THE PEAK GPU MEMORY OBSERVED DURING OPTIMIZATION.

PSNR $\uparrow$	Apple	Block	Paper	Spin	Teddy	Average	Time (s) $\downarrow$	Mem.(GB) $\downarrow$
4DGS [27]	24.74	22.13	22.65	24.35	19.27	22.63	14.649	9.72
STG [28]	25.70	21.96	23.93	23.95	18.02	22.71	4.181	2.71
MoSca [31]	22.55	21.91	22.86	22.54	19.61	21.89	9.238	34.68
SpeeDe3DGS [39]	22.03	20.03	22.29	21.57	17.29	20.64	5.989	33.68
ReCon-GS [42]	23.40	21.69	25.06	22.95	20.77	22.77	6.780	1.29
Instant4D [38]	26.51	23.70	25.31	24.86	21.04	24.28	1.413	16.39
Ours	27.17	23.65	26.14	26.07	21.12	24.83	0.432	2.18

TABLE II

QUANTITATIVE COMPARISON ON FOUR DAVIS SEQUENCES UNDER OUR UNIFIED PHOTOMETRIC-SUPERVISION PROTOCOL. WE REPORT PER-SCENE PSNR AND THE AVERAGE PSNR OVER THE FOUR SCENES. THE **BEST** AND **SECOND-BEST** RESULTS ARE HIGHLIGHTED. TIME AND MEM. DENOTE THE MEAN OPTIMIZATION TIME PER INPUT FRAME AND PEAK GPU MEMORY, RESPECTIVELY.

PSNR $\uparrow$	Bear	Breakdance	Rhino	Soapbox	Average	Time (s) $\downarrow$	Mem. (GB) $\downarrow$
4DGS [27]	19.25	19.89	19.83	16.17	18.78	68.167	7.65
STG [28]	21.45	20.40	21.75	18.75	20.59	4.844	2.25
MoSca [31]	24.10	21.82	24.66	21.71	23.07	26.041	18.96
SpeeDe3DGS [39]	25.90	20.26	27.62	17.05	22.71	17.385	18.98
ReCon-GS [42]	23.42	23.69	26.02	24.65	24.44	8.637	0.44
Instant4D [38]	29.24	20.71	30.18	22.25	25.29	2.406	3.47
Ours	27.51	20.87	29.79	27.60	26.44	0.932	0.58

Specifically, for each primitive $\bar{g}_i^j \in \bar{G}_i$ within the one cluster $\bar{G}_i$, we select the cluster center $g_i \in C_k$ as reference, and compute the image-plane offsets and their second-order spatial distribution as

$$\Delta \mathbf{u}_{ij} = \mathbf{u}(\bar{g}_i^j) - \mathbf{u}(g_i), \quad \mathbf{C}_i = \frac{1}{m_i} \sum_{j \in \mathcal{I}(\bar{G}_i)} \Delta \mathbf{u}_{ij} \Delta \mathbf{u}_{ij}^\top + \frac{1}{2} \mathbf{I}, \quad (6)$$

where $\mathbf{u}(\cdot)$ represents the centroid position of each primitive, $\mathcal{I}(\cdot)$ is the index set, $m_i = |\bar{G}_i|$, the term $\frac{1}{2} \mathbf{I}$ accounts for the spatial variance of the original one-pixel Gaussian footprint and prevents the covariance from becoming degenerate when the assigned candidates are closely concentrated.

Let $\lambda_{i,1} \geq \lambda_{i,2}$ be the eigenvalues of $\mathbf{C}_i$, and let $\mathbf{e}_{i,1}$ be the eigenvector associated with $\lambda_{i,1}$. We use the covariance to determine the principal direction and aspect ratio of the spatial region, and the linear anisotropy ratio is defined as $\rho_i = \min \left(\max \left(\sqrt{\frac{\lambda_{i,1}}{\lambda_{i,2}}}, 1 \right), |m_i| \right)$. The corresponding scale multipliers are $\gamma_{i,1} = \sqrt{m_i \rho_i}$, $\gamma_{i,2} = \sqrt{\frac{m_i}{\rho_i}}$.

Finally, we set the centroid position of $g(\bar{G}_i)$ as the centroid position of cluster center g_i , i.e., $\mathbf{u}(g(\bar{G}_i)) = \mathbf{u}(g_i)$, use the covariance matrix $\mathbf{C}_i$ to determine the rotation of $g(\bar{G}_i)$, and $\gamma_{i,1}, \gamma_{i,2}$ to be the scale of $g(\bar{G}_i)$, while leaving the motion parameter of $g(\bar{G}_i)$ to be the motion parameter of the cluster center g_i . In this way, we merge the primitives within one cluster $\bar{G}_i$ into a new representative primitive $g(\bar{G}_i)$. This primitive merging preserves the image-plane area associated with the represented candidates, and its in-plane orientation is also well aligned with $\mathbf{e}_{i,1}$ transformed into the world coordinate system.

D. Optimization

Finally, we perform a joint learning of compact 4DGS optimization for the compact static and dynamic Gaussian sets from monocular videos. Here since we have implemented a unified static/dynamic Gaussian splatting rasterization, at each timestep we can render the primitives together into a image given any camera views, and in turns use the rendered image as reference to perform the learning. Specifically, we use the rendered image $\hat{I}_t$ supervised by the corresponding input image I_t to formulate a combination of photometric and structural similarity losses:

$$L_{\text{rec}} = (1 - \lambda) L_1 + \lambda L_{\text{ssim}}, \quad (7)$$

where $L_1 = \|\hat{I}_t - I_t\|_1$, and $L_{\text{ssim}} = 1 - \text{SSIM}(\hat{I}_t, I_t)$, λ controls the balance between pixel-wise reconstruction accuracy and structural similarity.

IV. EXPERIMENTS

A. Implementation Details

Reprocessing. We estimate per-frame depth maps and camera poses using MegaSAM [65], compute optical flow between adjacent frames using SEA-RAFT [68], and obtain potential-motion masks using motion-guided SAM2 [66] propagation. The masks are propagated through the full sequence and combined over time so that an object remains assigned to the potential-motion component even when it is temporarily stationary. For MegaSAM, we optimize the consistent video depth for 200 iterations with an edge batch size of 128. SEA-RAFT is evaluated with 8 refinement iterations. Unless otherwise specified, all experiments use the same preprocessing and optimization settings.

Initialization and Pruning Configuration. For the temporally coherent initialization, we create temporal anchors for every 15 frames. Thus each anchor defines a local window with a radius of 15 frames, clipped at the sequence boundaries, and the valid 3D scene-flow tracklets in this window are used to initialize the dynamic primitive’s motion parameters. The dynamic initialization budget is set to 10% of the dense dynamic candidates. This budget is distributed among local motion groups in proportion to their candidate counts, while retaining at least one representative for every nonempty group. For the pruning processing, static candidates are back-projected at the same temporal anchors. Their target count is determined from the average static-to-potential-motion pixel ratio at these anchors, and the voxel size is selected automatically to approximately satisfy this target.

Optimization Details. We optimize each scene for 5,000 iterations using Adam with a batch size of one at the original input resolution. We follow the learning-rate schedule of the underlying 4D Gaussian implementation and set the structural-loss weight in Eq. 7 to $\lambda = 0.2$. Standard Gaussian refinement is enabled during optimization.

B. Datasets, Metrics and Baselines

Datasets. We evaluate Fast4DGS on two public monocular video datasets, including DyCheck iPhone [1], DAVIS [44], to perform the formal evaluation. Besides, we also use NeRF-DS [45] to demonstrate the generalization ability for our Fast4DGS. DyCheck iPhone contains casually captured dynamic scenes with moving cameras, object motion, and frequent occlusion changes. DAVIS provides diverse in-the-wild videos with complex foreground motion, nonrigid deformation, and challenging backgrounds. Since NeRF-DS provide two views video capture, we use one camera view captured video data for 4DGS optimization and perform the evaluation on the other camera view for comprehensive evaluation. Besides, we only focuses on dynamic indoor scenes with specular appearance, providing an additional challenge beyond predominantly diffuse content. Together, these datasets cover a broad range of camera trajectories, motion patterns, scene scales, and appearance conditions.

Specifically, we use dataset-specific evaluation protocols and apply the same split to all compared methods within each dataset. For DyCheck iPhone and DAVIS, frame 0 and frames whose indices are not divisible by 8 provide photometric supervision during optimization, while the remaining nonzero multiples of 8 are reserved for evaluation. These evaluation frames remain part of the input monocular video and may be used by the scene preprocessing stage, including geometry estimation, optical-flow computation, and potential-motion mask propagation, but they are excluded from the image reconstruction loss. For NeRF-DS, we train on the complete sequence from the left camera and evaluate on the temporally synchronized sequence from the right camera, thereby measuring novel-view synthesis. We render complete sequences for qualitative inspection and compute PSNR, SSIM, and LPIPS only on the corresponding evaluation frames or views. All experiments are conducted on a single NVIDIA A800

TABLE III
QUANTITATIVE COMPARISON ON NeRF-DS DATASET. PSNR, SSIM, AND LPIPS ARE AVERAGED OVER THE EVALUATED SCENES. THE **BEST** AND **SECOND-BEST** RESULTS ARE HIGHLIGHTED. TIME DENOTES THE MEAN TRAINING TIME (SECONDS PER-FRAME). MEM. IS THE MAXIMUM GPU MEMORY OBSERVED DURING OPTIMIZATION.

Methods	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Time $\downarrow$	Mem $\downarrow$
4DGS [27]	21.26	0.771	0.198	3.227	3.50
STG [28]	22.57	0.821	0.165	0.636	9.11
MoSca [31]	21.47	0.789	0.274	2.843	6.15
SpeeDe3DGS [39]	23.58	0.846	0.097	1.882	6.27
ReCon-GS [42]	21.33	0.781	0.210	2.198	0.73
Instant4D [38]	21.54	0.801	0.210	0.325	3.86
Ours	22.69	0.823	0.119	0.194	1.04

GPU with 80 GB of memory. Training time and peak GPU memory are measured using the same hardware and software environment for all the methods.

Metrics. We evaluate reconstruction quality using PSNR, SSIM, and LPIPS on the corresponding evaluation frames or views. The DyCheck iPhone and DAVIS experiments measure reconstruction quality on frames excluded from photometric supervision, whereas NeRF-DS evaluates novel-view synthesis from the synchronized right camera. To evaluate efficiency, we report optimization time and peak GPU memory. All timing comparisons are measured under a common hardware environment.

Baselines. We mainly compare with previous dynamic reconstruction approaches using 4D Gaussian splatting, including: 4DGS [27], STG [28], Instant4D [38], SpeeDe3DGS [39], and MoSca [31]. MoSca is a pioneering monocular 4DGS reconstruction, which we use it as representative approach for recent state-of-the-art monocular 4DGS reconstruction. Instant4D and SpeeDe3DGS target to accelerate the monocular 4DGS reconstruction as ours, which are two state-of-the-art 4DGS acceleration approaches. 4DGS and STG are not for monocular 4DGS reconstruction, but we use them as baselines for dynamic quality evaluation. Besides, we don’t directly compare with 4D-Fly [40] since they didn’t release the source code, but we use ReCon-GS [42] as representative though them aim at multiview 4DGS reconstruction. For fair comparison, we use the default configuration as they provided in the public released source code. For SpeeDe3DGS¹, we perform the comparison using only one GPU device as like other baselines for fair comparison, even though they can be faster using multiple GPU acceleration.

C. Quantitative Evaluation

Reconstruction quality. Tables I, II, and III report the quantitative comparisons on DyCheck iPhone, DAVIS, and NeRF-DS Datasets respectively. On the five DyCheck iPhone sequences, Fast4DGS achieves the best reconstruction quality with an average PSNR of 24.83 dB, which outperforms the second best alternative Instant4D with more than 0.55dB PSNR improvement, and consistently better than other baselines. Specifically, Fast4DGS obtains the best result on four

¹<https://speede3dgs.github.io/>

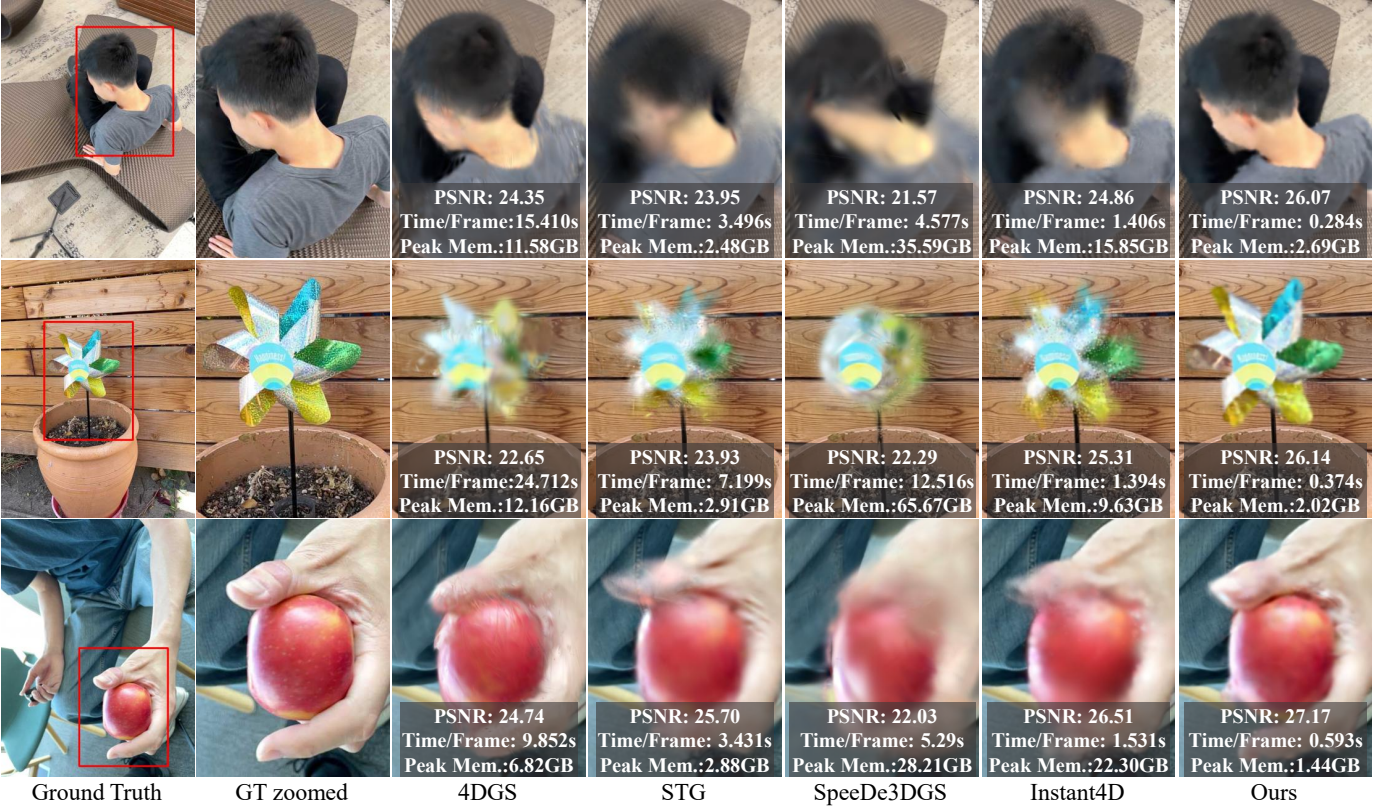


Fig. 5. Qualitative comparison on representative DyCheck iPhone sequences. From top to bottom, the rows show *spin*, *paper-windmill*, and *apple*. The red boxes in the ground-truth images indicate the enlarged dynamic regions shown in the remaining columns. For each method, we additionally report the per-scene PSNR, optimization time per input frame, and peak GPU memory.

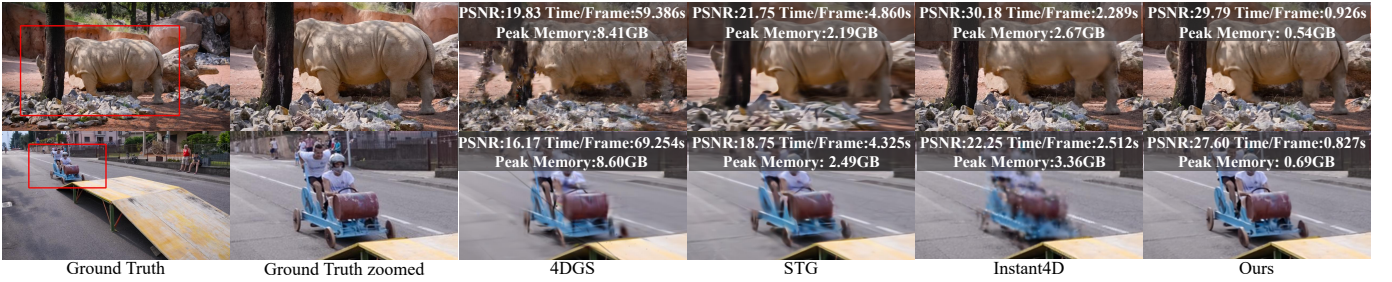


Fig. 6. Qualitative comparison on representative DAVIS sequences. From top to bottom, the rows show *rhino* and *soapbox*. The red boxes in the ground-truth images indicate the enlarged dynamic regions shown in the remaining columns. For each method, we additionally report the per-scene PSNR, optimization time per input frame, and peak GPU memory.

of the five sequences and remains comparable on *block* with Instant4D. On DAVIS dataset evaluation, Fast4DGS also achieves the best reconstruction quality with average 26.44dB PSNR, which is consistently better than Instant4D, Recon-GS, Speede3DGS and other baselines. Similarly, on NeRF-DS, Fast4DGS achieves 22.69 dB PSNR, 0.823 SSIM, and 0.119 LPIPS, which outperforms 4DGS, STG, and Instant4D baselines. Since NeRF-DS provides ground truth camera poses, Speede3DGS achieves the best dynamic reconstruction quality when using the ground truth camera poses. Besides, the moving region in NeRF-DS is also small which is suitable for the GroupFlow of Speede3DGS, thus leading to the best performance. But Fast4DGS still outperforms other baselines.

Training efficiency. For the training speed comparison,

Fast4DGS requires average 0.432 seconds per-frame training time on DyCheck, which is $3.27\times$ faster than Instant4D, and much faster than other baselines. For DAVIS dataset, Fast4DGS achieves average 0.932 seconds per-frame training time, which is $2.58\times$ faster than Instant4D, and also much faster than other baselines. Similarly, on NeRF-DS, Fast4DGS reduces the optimization time to 0.194 seconds per frame, corresponding to a $1.68\times$ speedup over Instant4D and a $3.28\times$ speedup over STG. These timings exclude reprocessing and measure per-scene optimization only.

Memory efficiency. In general, Fast4DGS uses about 2.18GB of peak GPU memory on DyCheck, compared with 16.39GB for Instant4D, which is also consistently lower than other baselines. For DAVIS dataset, Fast4DGS uses 0.58GB of

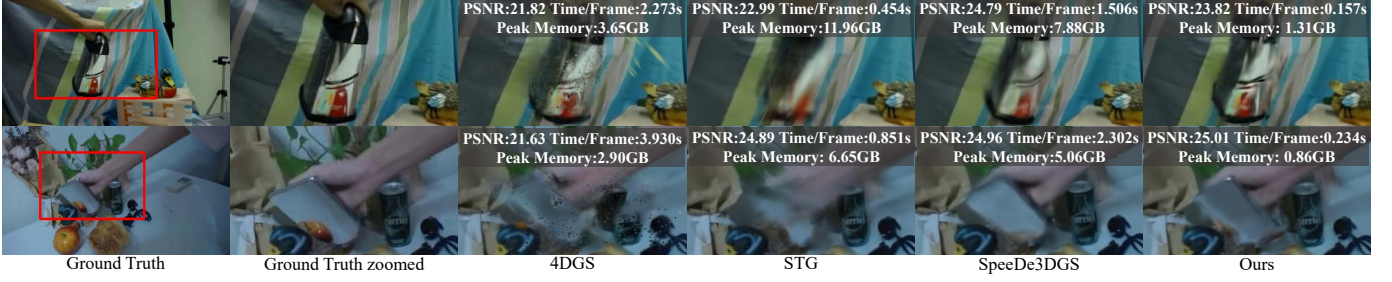


Fig. 7. Qualitative comparison on NeRF-DS under the left-camera training and synchronized right-camera evaluation protocol. The red boxes indicate the enlarged regions. Fast4DGS preserves coherent moving-object structures and sharper motion boundaries, while the competing methods exhibit motion blur, floating artifacts, or local structural degradation.

peak GPU memory consumption, which is slightly higher than ReCon-GS but are consistently lower than other baselines. On NeRF-DS, Fast4DGS requires 1.04GB, substantially less than all compared baselines.

D. Qualitative Evaluation

We also perform qualitative evaluation on the visual comparison results of these comparing approaches. Fig. 5 compares Fast4DGS with the other baselines on challenging dynamic regions of DyCheck. In *spin*, Fast4DGS better preserves the subject’s hair silhouette and clothing boundary, with substantially less motion-induced smearing. In *paper-windmill*, our method reconstructs the individual blades and their color pattern, whereas the competing methods tend to merge the rapidly rotating structure into blurred or translucent regions. In *apple*, Fast4DGS retains a more coherent apple shape and cleaner hand–object boundaries with fewer streaking artifacts. These visual improvements are consistent with the higher PSNR achieved by Fast4DGS on all three examples.

Fig. 6 presents qualitative comparisons on the DAVIS sequences, which contain large foreground motion and complex nonrigid dynamics. In *rhino*, Fast4DGS recovers a coherent animal silhouette and preserves the body structure with substantially fewer floaters than Instant4D and less motion-induced smearing than 4DGS and STG. In *soapbox*, our method reconstructs the riders and the rapidly moving cart with sharper boundaries and clearer internal structure. In contrast, 4DGS and STG produce strong motion blur, while Instant4D exhibits severe floating artifacts. These results show that Fast4DGS remains robust on challenging in-the-wild motion while requiring considerably less optimization time and GPU memory.

Fig. 7 extends the qualitative comparison to cross-camera novel-view synthesis on NeRF-DS, where only the left-camera sequence is used for training and the synchronized right-camera views are reserved for evaluation. In *press*, Fast4DGS maintains a coherent hand contour and a recognizable pressing tool, whereas STG exhibits pronounced motion blur and 4DGS and Instant4D produce floating artifacts around the moving region. In *bell*, our method better preserves the silhouette and internal structure of the moving bell, while the competing reconstructions show stronger blurring, fragmented boundaries, or translucent artifacts. Please refer to our supplementary materials and demo video for more results.

E. Ablation Studies

We conduct ablation studies on the main components used in our Fast4DGS, including the motion-aware initialization, locally compact pruning. For each component, we also further investigate the impact of sub-modules on the final reconstruction speed and quality. We perform the ablation studies on *apple* and *paper-windmill*, which contain different camera trajectories and motion patterns.

Motion-aware Initialization. We first study the motion-aware initialization by evaluating variants without using the overall initialization (termed as ‘w/o Initialization’), temporal coherent initialization (termed as ‘w/o coherent’) and learnable temporal window (termed as ‘w/o learnable window’). For ‘w/o Initialization’, we implemented a variant without using our motion-aware initialization, but with a random initialization which amounts to learn C-4DGS primitives from scratch. For variant ‘w/o coherent’, we perform naive motion parameter fitting by simply estimating the motion parameters of dynamic primitives according to Eq. (4) within the whole range.

Locally Compact Pruning. We then study the locally compact pruning by implementing variants without using the overall pruning (termed as ‘w/o Pruning’), the primitive merging (termed as ‘w/o merging’) and anisotropic preservation in the primitive merging (termed as ‘w/o anisotropic’). For variant ‘w/o Pruning’, we didn’t perform the pruning at all during the overall training stage. For variant ‘w/o merging’, we implement a naive representative primitive selection by simply setting $g(\vec{G}_i) = g_i \in C_k$. For variant ‘w/o anisotropic’, we implement the primitive merging without considering the covariance matrix adaptation but simply keeping it as the covariance matrix from the cluster center.

The ablation studies results are shown in Table IV. As we can see, without using motion-aware initialization, ‘w/o Initialization’ achieves much slower training speed than our full system, and also gets dynamic reconstruction quality decrease. Similar results are also shown for variant ‘w/o Pruning’, which reduces the optimization time from 0.879 to 0.483 seconds per frame while improving PSNR by 0.31 dB. Besides, ‘w/o coherent’ provides the largest quality improvement, raising the average PSNR from 23.87 to 26.66 dB. ‘w/o learnable window’ further improves PSNR by 1.51 dB. ‘w/o anisotropic’ and ‘w/o merging’ footprint adjustment mainly improve reconstruction quality, with only minor changes in



Fig. 8. Qualitative component ablation on *paper-windmill* (top) and *apple* (bottom). From left to right, we show the ground-truth frame, the enlarged ground-truth region, five ablated variants, and the full model. All variants are visualized using the same crop. Local cubic fitting is essential for recovering rapidly moving structures, while the temporal-window and coverage-preservation components further reduce blur, boundary artifacts, and floaters.

TABLE IV
COMPONENT-WISE ABLATION OF FAST4DGS. RESULTS ARE AVERAGED OVER *apple* AND *paper-windmill*. EACH VARIANT IS DERIVED FROM THE FULL MODEL BY REMOVING OR REPLACING ONLY THE INDICATED COMPONENT. TIME REPORTS THE TRAINING SPEED IN TERMS OF SECONDS PER-FRAME.

Methods	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Time $\downarrow$
w/o Initialization	25.52	0.837	0.150	0.894
w/o Pruning	26.35	0.838	0.143	0.879
w/o coherent	23.87	0.831	0.167	0.447
w/o learnable window	25.15	0.835	0.144	0.486
w/o anisotropic	26.53	0.842	0.138	0.480
w/o merging	26.26	0.839	0.137	0.470
Ours (Full)	26.66	0.842	0.137	0.483

optimization time. Fig. 8 show the visual comparison results between these different variants.

F. Parameter Sensitivity

We analyze the sensitivity of Fast4DGS to the key parameters configurations used in temporal-anchor interval (for temporal coherent initialization), local window radius ((for temporal coherent initialization)), and dynamic pruning ratio (for the dynamic primitive pruning). We conduct the experiments on *apple* and *paper-windmill*, following the same evaluation protocol as the component ablation. The results are reported in Table V.

Temporal-anchor interval. A shorter anchor interval creates more overlapping local groups and provides a finer temporal partition for approximating non-rigid motion. Reducing the interval from 15 to 10 frames improves PSNR by 0.04 dB and LPIPS from 0.137 to 0.133, but increases time per frame by approximately 9.1% and peak memory by 19.7% because both the dynamic initialization and the coupled static initialization become denser. In contrast, an interval of 20 frames reduces the number of local groups and lowers the computational cost, but degrades PSNR by 0.14 dB and LPIPS to 0.140, indicating that the coarser temporal partition is less effective at representing rapidly changing motion. We

therefore use 15 frames as a balanced operating point: the 10-frame setting provides only a modest quality gain at a noticeably higher resource cost, whereas the 20-frame setting sacrifices reconstruction quality.

Local window radius. The window radius controls the temporal window used by each local cubic trajectory without changing the number of anchors or the initialization budget. Consequently, its influence on efficiency is small: all three settings require approximately 0.47–0.49 seconds per-frame and 1.72–1.73 GB of peak memory. A radius of 10 frames provides insufficient temporal evidence for stable trajectory fitting, reducing PSNR by 0.13 dB relative to the default. Increasing the radius to 20 frames improves temporal coverage but includes a broader range of motion within each local fit, resulting in a 0.08 dB PSNR reduction and slightly worse LPIPS. The 15-frame radius therefore provides the best balance between local fitting accuracy and temporal coverage.

Dynamic pruning ratio. The dynamic pruning ratio directly controls the capacity of the primitive numbers. Reducing the dynamic pruning ratio from 10% to 5% decreases time per frame by approximately 9.9% and peak memory by 19.1%, but reduces PSNR by 0.04 dB and increases LPIPS from 0.137 to 0.147. This indicates that an overly compact initialization mainly removes fine motion-dependent details that are more clearly reflected by the perceptual metric. Increasing the dynamic pruning ratio to 15%, 20% improves LPIPS to 0.133, 0.132 and SSIM to 0.846, 0.847 respectively, but provides no significant PSNR gain while increasing time per-frame by 23.6% and peak memory by 45.1%. These diminishing returns support the 10% dynamic pruning ratio (we used in the system) as an effective compromise between reconstruction quality and efficiency.

V. CONCLUSION

This paper proposes a novel monocular 4DGS reconstruction approach, called Fast4DGS, which targets at much faster 4DGS learning from monocular videos. Our key idea is to build up compact 4DGS primitive representation with motion-aware initialization, and further reduce the temporal redundancy with a locally compact pruning. By Extensive eval-

TABLE V
PARAMETER SENSITIVITY OF FAST4DGS. RESULTS ARE AVERAGED OVER *apple* AND *paper-windmill*. DEFAULT SETTINGS ARE SHOWN IN BOLD.

Parameter	Value	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	Time (s) $\downarrow$	Mem. (GB) $\downarrow$
Anchor interval	10 frames	26.70	0.848	0.133	0.527	2.07
	20 frames	26.52	0.843	0.140	0.469	1.53
Window radius	10 frames	26.53	0.845	0.138	0.474	1.72
	20 frames	26.58	0.846	0.138	0.485	1.72
Dynamic pruning ratio	5%	26.62	0.840	0.147	0.435	1.40
	15%	26.69	0.846	0.133	0.580	2.08
	20%	26.66	0.847	0.132	0.597	2.51
Ours	15/15/10%	26.66	0.842	0.137	0.483	1.73

uation on DyCheck iPhone, DAVIS and NeRF-DS datasets, our Fast4DGS consistently outperform state-of-the-art 4DGS acceleration approaches with faster training speed and better or competitive dynamic rendering quality.

One main limitation of our Fast4DGS is reconstruction quality degradation for fast motion dynamic scenes. This is primarily because the priors relied on by our Fast4DGS, including depth, camera pose and optical flow, degrade substantially in fast-motion scenarios. We leave this for future work, and hope that Fast4DGS could inspire many subsequent efforts for more efficient while high fidelity monocular 4DGS reconstruction.

REFERENCES

- [1] H. Gao, R. Li, S. Tulsiani, B. Russell, and A. Kanazawa, "Monocular dynamic view synthesis: A reality check," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 33 768–33 780.
- [2] A. Mustafa, H. Kim, J.-Y. Guillemaut, and A. Hilton, "General dynamic scene reconstruction from multiple view video," in *Proceedings of the IEEE International Conference on Computer Vision*, 2015, pp. 900–908.
- [3] M.-F. Yu, L. Zhang, W.-F. Wang, and J.-H. Wang, "Scp-slam: Accelerating dynaslam with static confidence propagation," in *2023 IEEE Conference Virtual Reality and 3D User Interfaces (VR)*, 2023, pp. 509–518.
- [4] M. Zhao, S. Nag, K. Wang, A. Vora, G. Ji, P. Chun, A. Mahdavi-Amiri, and H. Zhang, "Advances in 4d representation: Geometry, motion, and interaction," *Computer Graphics Forum*, 2026.
- [5] J. Kim, T. Jin, M. Dasari, S. Seshan, and A. Rowe, "Scenehub4d: A dataset and evaluation framework for 6-dof 4d vr scenes," *IEEE Transactions on Visualization and Computer Graphics*, vol. 32, no. 5, pp. 4143–4153, 2026.
- [6] Y. Zhang, J. Yang, Z. Liu, R. Wang, G. Chen, X. Tong, and B. Guo, "Virtualcube: An immersive 3d video communication system," *IEEE TVCG*, vol. 28, no. 5, pp. 2146–2156, 2022.
- [7] H. Tu, R. Shao, X. Dong, S. Zheng, H. Zhang, L. Chen, M. Wang, W. Li, S. Ma, S. Zhang *et al.*, "Tele-aloha: A telepresence system with low-budget and high-authenticity using sparse rgb cameras," in *ACM SIGGRAPH Conference Papers*, 2024, pp. 1–12.
- [8] Y. Shen, W. Liang, and J. Ma, "Liteat: A data-lightweight and user-adaptive vr telepresence system for remote education," *IEEE TVCG*, 2025.
- [9] G. Gafni, J. Thies, M. Zollhöfer, and M. Nießner, "Dynamic neural radiance fields for monocular 4d facial avatar reconstruction," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.
- [10] L. Zhao, X. Lu, R. Fan, S. K. Im, and L. Wang, "Gaussianhand: Real-time 3d gaussian rendering for hand avatar animation," *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, no. 9, pp. 6484–6496, 2025.
- [11] J. Chen, Y. Zhang, R. Pandey, T. Beeler, M. Habermann, and C. Theobalt, "Egorelight: Egocentric human capture and illumination recovery for relightable and photoreal avatar rendering," *ACM Transactions on Graphics (TOG)*, vol. 45, no. 4, pp. 1–19, 2026.
- [12] L. Song, A. Chen, Z. Li, Z. Chen, L. Chen, J. Yuan, Y. Xu, and A. Geiger, "Nerfplayer: A streamable dynamic scene representation with decomposed neural radiance fields," *IEEE Transactions on Visualization and Computer Graphics*, vol. 29, no. 5, pp. 2732–2742, 2023.
- [13] J. Kim, M. Stengel, A. Mazumdar, T. Li, C. Sun, D. Luebke, and S. De Mello, "Play4d: Accelerated and interactive free-viewpoint video streaming for virtual reality and light field displays," in *SIGGRAPH Asia 2025 Emerging Technologies*, December 2025.
- [14] S. Lee, S. Park, and H. Kim, "Dynscene: Scalable generation of dynamic robotic manipulation scenes for embodied ai," in *IEEE CVPR*, 2025, pp. 12 166–12 175.
- [15] Y. Jiang, C. Yu, T. Xie, X. Li, Y. Feng, H. Wang, M. Li, H. Lau, F. Gao, Y. Yang *et al.*, "Vr-gs: A physical dynamics-aware interactive gaussian splatting system in virtual reality," in *ACM SIGGRAPH conference papers*, 2024, pp. 1–1.
- [16] M. Alfonso-Arsuaga, H. Dominguez-Elvira, J. Casas-Guerrero, A. Castiella-Aguirrezabala, L. C. Domínguez, J. García-González, M. Naranjo-Almeida, M. Comino-Trinidad, and J. Lopez-Moreno, "Physplat: A framework for photorealistic hybrid simulation of real and synthetic elements using 3d gaussian splatting," in *IEEE WACV*, 2026, pp. 1621–1631.
- [17] W. Wang, J. Xiao, Y. Yang, Y. Zhuang, and L. Chen, "Physically plausible human-object rendering from sparse views via 3d gaussian splatting," *IEEE TIP*, 2026.
- [18] T. Li, M. Slavcheva, M. Zollhoefer, S. Green, C. Lassner, C. Kim, T. Schmidt, S. Lovegrove, M. Goesele, R. Newcombe *et al.*, "Neural 3d video synthesis from multi-view video," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2022, pp. 5511–5521.
- [19] A. Cao and J. Johnson, "Hexplane: A fast representation for dynamic scenes," in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2023, pp. 130–141.
- [20] D. Li, S.-S. Huang, T. Shen, and H. Huang, "Dynamic view synthesis with spatio-temporal feature warping from sparse views," in *Proceedings of the 31st ACM International Conference on Multimedia*, 2023, pp. 1565–1576.
- [21] D. Li, S.-S. Huang, Z. Lu, X. Duan, and H. Huang, "St-4dgs: Spatial-temporally consistent 4d gaussian splatting for efficient dynamic scene rendering," in *ACM SIGGRAPH 2024 conference papers*, 2024, pp. 1–11.
- [22] C. Gao, A. Saraf, J. Kopf, and J.-B. Huang, "Dynamic view synthesis from dynamic monocular video," in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 5692–5701.
- [23] A. Pumarola, E. Corona, G. Pons-Moll, and F. Moreno-Noguer, "D-nerf: Neural radiance fields for dynamic scenes," in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2021, pp. 10 313–10 322.
- [24] J. Fang, T. Yi, X. Wang, L. Xie, X. Zhang, W. Liu, M. Nießner, and Q. Tian, "Fast dynamic radiance fields with time-aware neural voxels," in *SIGGRAPH Asia 2022 conference papers*, 2022, pp. 1–9.
- [25] J. Zhang, X. Liu, X. Ye, F. Zhao, Y. Zhang, M. Wu, Y. Zhang, L. Xu, and J. Yu, "Editable free-viewpoint video using a layered neural representation," *ACM Transactions on Graphics (TOG)*, vol. 40, no. 4, pp. 1–18, 2021.
- [26] B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis *et al.*, "3d gaussian splatting for real-time radiance field rendering," *ACM TOG*, vol. 42, no. 4, pp. 139–1, 2023.
- [27] G. Wu, T. Yi, J. Fang, L. Xie, X. Zhang, W. Wei, W. Liu, Q. Tian, and X. Wang, "4d gaussian splatting for real-time dynamic scene

- rendering,” in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2024, pp. 20310–20320.
- [28] Z. Li, Z. Chen, Z. Li, and Y. Xu, “Spacetime gaussian feature splatting for real-time dynamic view synthesis,” in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2024, pp. 8508–8520.
- [29] Y. Wang, P. Yang, Z. Xu, J. Sun, Z. Zhang, Y. Chen, H. Bao, S. Peng, and X. Zhou, “Freetimegs: Free gaussian primitives at anytime anywhere for dynamic scene reconstruction,” in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2025, pp. 21750–21760.
- [30] Q. Wang, V. Ye, H. Gao, W. Zeng, J. Austin, Z. Li, and A. Kanazawa, “Shape of motion: 4d reconstruction from a single video,” in *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 2025, pp. 9660–9672.
- [31] J. Lei, Y. Weng, A. W. Harley, L. Guibas, and K. Daniilidis, “Mosca: Dynamic gaussian fusion from casual videos via 4d motion scaffolds,” in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2025, pp. 6165–6177.
- [32] Q. Liu, Y. Liu, J. Wang, X. Lyu, P. Wang, W. Wang, and J. Hou, “Modgs: Dynamic gaussian splatting from casually-captured monocular videos with depth priors,” in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 97048–97074.
- [33] M.-L. Shih, Y.-H. Chen, Y.-L. Liu, and B. Curless, “Prior-enhanced gaussian splatting for dynamic scene reconstruction from casual video,” in *Proceedings of the SIGGRAPH Asia 2025 Conference Papers*, 2025, pp. 1–13.
- [34] C. Zhang, G. Le Moing, S. Koppula, I. Rocco, L. Momeni, J. Xie, S. Sun, R. Sukthankar, J. K. Barral, R. Hadsell *et al.*, “Efficiently reconstructing dynamic scenes one d4rt at a time,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 7382–7392.
- [35] J. Karhade, N. Keetha, Y. Zhang, T. Gupta, A. Sharma, S. Scherer, and D. Ramanan, “Any4d: Unified feed-forward metric 4d reconstruction,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 14578–14589.
- [36] Z. Xu, Z. Li, Z. Dong, X. Zhou, R. Newcombe, and Z. Lv, “4dgt: Learning a 4d gaussian transformer using real-world monocular videos,” in *Advances in Neural Information Processing Systems*, D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, Eds., vol. 38, Main Conference. Curran Associates, Inc., 2025, pp. 42606–42632.
- [37] C. Lin, Y. Lin, P. Pan, Y. Yu, T. Hu, H. Yan, K. Fragkiadaki, and Y. Mu, “Movies: Motion-aware 4d dynamic view synthesis in one second,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 295–306.
- [38] Z. Luo, H. Ran, and L. Lu, “Instant4d: 4d gaussian splatting in minutes,” *Advances in neural information processing systems*, vol. 38, pp. 29953–29972, 2026.
- [39] A. Tu, H. Ying, A. Hanson, Y. Lee, T. Goldstein, and M. Zwicker, “Speede3dgs: Speedy deformable 3d gaussian splatting with temporal pruning and motion grouping,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 26083–26093.
- [40] D. Wu, F. Liu, Y.-H. Hung, Y. Qian, X. Zhan, and Y. Duan, “4d-fly: Fast 4d reconstruction from a single monocular video,” in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2025, pp. 16663–16673.
- [41] Z. Liu, Y. Hu, X. Zhang, R. Song, J. Shao, Z. Lin, and J. Zhang, “Dynamics-aware gaussian splatting streaming towards fast on-the-fly 4d reconstruction,” *IEEE Transactions on Visualization and Computer Graphics*, 2026.
- [42] J. Fu, Q. Gao, C. Wen, Y. Wu, S. Ma, J. Zhang, and J. Zhang, “Recongs: Continuum-preserved gaussian streaming for fast and compact reconstruction of dynamic scenes,” *Advances in Neural Information Processing Systems*, vol. 38, pp. 134751–134778, 2026.
- [43] D. Yin, I. Ding, Y. Jin, J. Shi, and J. Liu, “Trackersplat: Exploiting point tracking for fast and robust dynamic 3d gaussians reconstruction,” in *Proceedings of the SIGGRAPH Asia 2025 Conference Papers*, 2025, pp. 1–11.
- [44] F. Perazzi, J. Pont-Tuset, B. McWilliams, L. Van Gool, M. Gross, and A. Sorkine-Hornung, “A benchmark dataset and evaluation methodology for video object segmentation,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 724–732.
- [45] Z. Yan, C. Li, and G. H. Lee, “Nerf-ds: Neural radiance fields for dynamic specular objects,” *CVPR*, 2023.
- [46] R. A. Newcombe, D. Fox, and S. M. Seitz, “Dynamicfusion: Reconstruction and tracking of non-rigid scenes in real-time,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 343–352.
- [47] M. Loper, N. Mahmood, J. Romero, G. Pons-Moll, and M. J. Black, “Smpl: A skinned multi-person linear model,” in *Seminal Graphics Papers: Pushing the Boundaries, Volume 2*, 2023, pp. 851–866.
- [48] T. Yu, Z. Zheng, K. Guo, J. Zhao, Q. Dai, H. Li, G. Pons-Moll, and Y. Liu, “Doublefusion: Real-time capture of human performances with inner body shapes from a single depth sensor,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 7287–7296.
- [49] T. Yu, Z. Zheng, K. Guo, P. Liu, Q. Dai, and Y. Liu, “Function4d: Real-time human volumetric capture from very sparse consumer rgbd sensors,” in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2021, pp. 5742–5752.
- [50] C. L. Zitnick, S. B. Kang, M. Uyttendaele, S. Winder, and R. Szeliski, “High-quality video view interpolation using a layered representation,” *ACM transactions on graphics (TOG)*, vol. 23, no. 3, pp. 600–608, 2004.
- [51] A. Cao, C. Rockwell, and J. Johnson, “Fwd: Real-time novel view synthesis with forward warping and depth,” in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 15692–15703.
- [52] Y. Zhou, T. Yu, Z. Zheng, G. Wu, G. Zhao, W. Jiang, Y. Fu, and Y. Liu, “Probibr: Fast image-based rendering with learned probability-guided sampling,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, no. 3, pp. 1888–1901, 2025.
- [53] R. Martin-Brualla, N. Radwan, M. S. Sajjadi, J. T. Barron, A. Dosovitskiy, and D. Duckworth, “Nerf in the wild: Neural radiance fields for unconstrained photo collections,” in *2021 IEEE/CVF conference on computer vision and pattern recognition (CVPR)*. IEEE, 2021, pp. 7206–7215.
- [54] K. Park, U. Sinha, P. Hedman, J. T. Barron, S. Bouaziz, D. B. Goldman, R. Martin-Brualla, and S. M. Seitz, “Hypernerf: A higher-dimensional representation for topologically varying neural radiance fields,” *ACM Trans. Graph.*, vol. 40, no. 6, dec 2021.
- [55] R. Shao, Z. Zheng, H. Tu, B. Liu, H. Zhang, and Y. Liu, “Tensor4d: Efficient neural 4d decomposition for high-fidelity dynamic reconstruction and rendering,” in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 16632–16642.
- [56] Y. Jiang, C. Yu, T. Xie, X. Li, Y. Feng, H. Wang, M. Li, H. Lau, F. Gao, Y. Yang, and C. Jiang, “Vr-gs: A physical dynamics-aware interactive gaussian splatting system in virtual reality,” in *ACM SIGGRAPH 2024 Conference Papers*, ser. SIGGRAPH ’24, 2024.
- [57] R. Fan, J. Wu, X. Shi, L. Zhao, Q. Ma, and L. Wang, “Fov-gs: Foveated 3d gaussian splatting for dynamic scenes,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, no. 5, pp. 2975–2985, 2025.
- [58] K. Cheng, K. Yang, X. Long, X. Guo, and X. Chen, “Hie4dgs: Hierarchical 4d gaussian splatting from monocular dynamic video,” *IEEE Transactions on Visualization and Computer Graphics*, 2026.
- [59] Y. Duan, F. Wei, Q. Dai, Y. He, W. Chen, and B. Chen, “4d-rotor gaussian splatting: Towards efficient novel view synthesis for dynamic scenes,” in *ACM SIGGRAPH 2024 Conference Papers*, ser. SIGGRAPH ’24, 2024.
- [60] Y. Zhao, H. Zhang, K. Huang, Q. Wang, Z. Shao, and D. Wu, “Ru4d-slam: Reweighting uncertainty in gaussian splatting slam for 4d scene reconstruction,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Findings*, June 2026, pp. 1492–1502.
- [61] Y. Liang, R. Zhu, H. Chang, Z. Li, J. Lu, and T. Zhang, “Reflow: Self-correction motion learning for dynamic scene reconstruction,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2026, pp. 29803–29813.
- [62] S. Morkva, V. Patil, A. Tonioni, M. Oechsle, M. Wilder-Smith, and M. Hutter, “Mosaic-gs: Monocular scene reconstruction via advanced initialization for complex dynamic environments,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2026, pp. 1167–1176.
- [63] A. Hanson, A. Tu, V. Singla, M. Jayawardhana, M. Zwicker, and T. Goldstein, “Pup 3d-gs: Principled uncertainty pruning for 3d gaussian splatting,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2025, pp. 5949–5958.
- [64] A. Hanson, A. Tu, G. Lin, V. Singla, M. Zwicker, and T. Goldstein, “Speedy-splat: Fast 3d gaussian splatting with sparse pixels and sparse primitives,” in *Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR)*, June 2025, pp. 21537–21546.

- [65] Z. Li, R. Tucker, F. Cole, Q. Wang, L. Jin, V. Ye, A. Kanazawa, A. Holynski, and N. Snavely, “Megaseam: Accurate, fast and robust structure and motion from casual dynamic videos,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2025, pp. 10 486–10 496.
- [66] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár, and C. Feichtenhofer, “Sam 2: Segment anything in images and videos,” in *International Conference on Learning Representations*, 2025.
- [67] L. Kaufman and P. J. Rousseeuw, *Finding Groups in Data: An Introduction to Cluster Analysis*. John Wiley & Sons, 1990, ch. 2, pp. 68–125.
- [68] Y. Wang, L. Lipson, and J. Deng, “Sea-raft: Simple, efficient, accurate raft for optical flow,” in *European Conference on Computer Vision*. Springer, 2024, pp. 36–54.

For DAVIS dataset in Fig. 10 and Fig. 12, and NeRF-DS dataset in Fig. 13, our Fast4DGS also similarly achieves better performance than those previous 4DGS baselines. Please refer to the supplementary video for more visual comparison results.

APPENDIX A

The appendices provide more visual comparison results between different comparing approaches including 4DGS, STG, Speede3DGS, Instant4D and our Fast4DGS, evaluated on DyCheck iPhone, DAVIS and NeRF-DS dataset. For the DyCheck iPhone dataset in Fig. 9, our Fast4DGS can achieve more accurate dynamic reconstruction with fewer artifacts, while at much faster training speed and lower Peak GPU memory consumption. This is corroborated by the visual comparison from novel view rendering shown in Fig. 11.

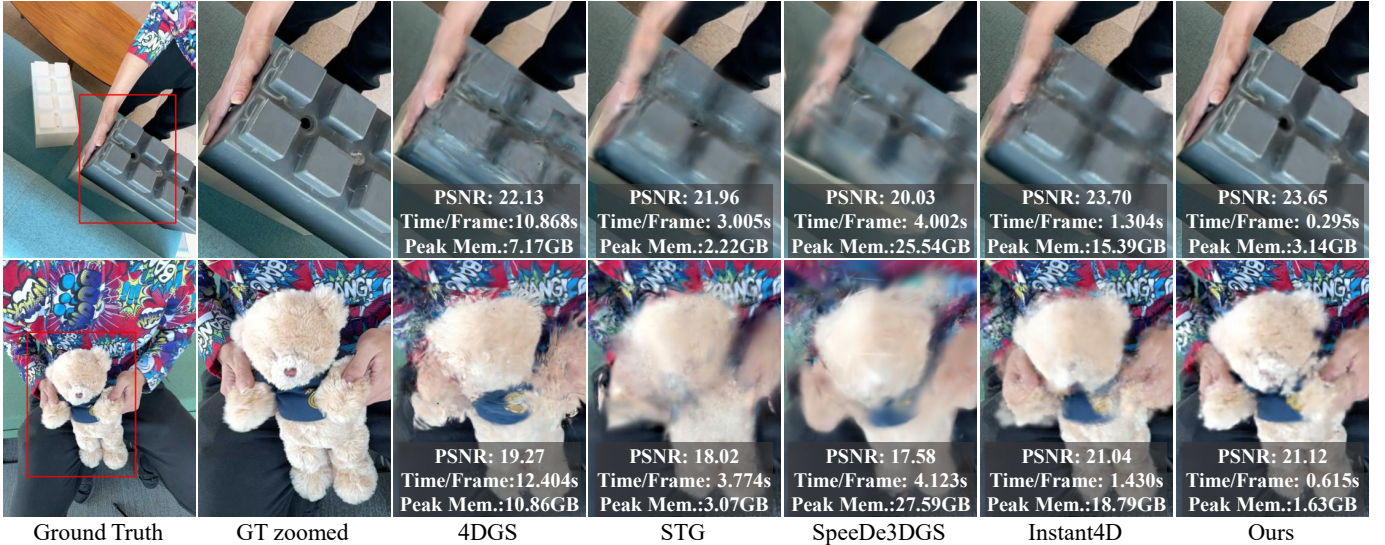


Fig. 9. Visual comparison results evaluated on DyCheck iPhone. From top to bottom: *block* and *teddy*. The crops emphasize hand–object boundaries and non-rigid appearance under motion. The metrics are obtained by averaging across the image sequence.

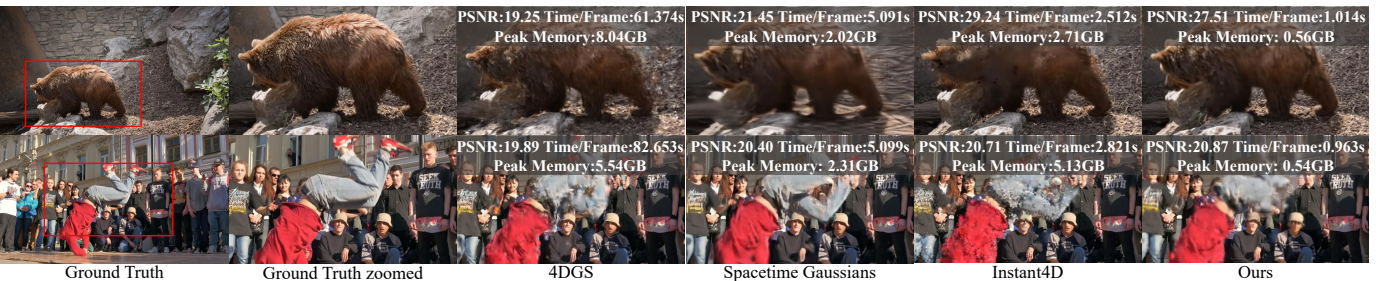


Fig. 10. Visual comparison results evaluated on DAVIS. From top to bottom: *bear* and *breakdance*. The comparisons cover articulated animal motion and fast human motion. The metrics are obtained by averaging across the image sequence.



Fig. 11. Visual comparison results evaluated on DyCheck iPhone. The crops emphasize hand-object boundaries and non-rigid appearance under motion. The metrics are obtained by averaging across the image sequence.



Fig. 12. Visual comparison results evaluated on DAVIS. The comparisons cover articulated animal motion and fast human motion. The metrics are obtained by averaging across the image sequence.

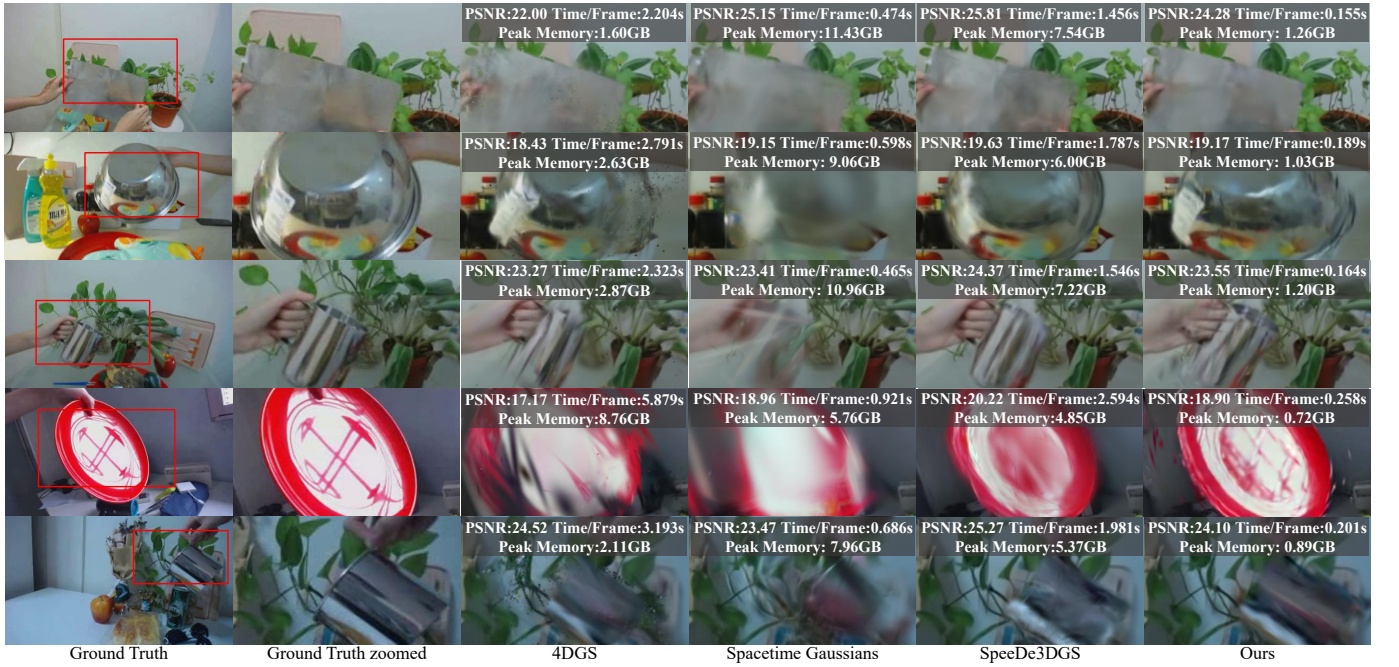


Fig. 13. Visual comparison results evaluated on NeRF-DS. From top to bottom: *as*, *basin*, *cup*, *plate*, and *sieve*. The selected regions contain moving boundaries, reflections, thin structures, and rapid object motion. The metrics are obtained by averaging across the image sequence.